

## PRD md

### # AI Lecture Notes Generator — PRD

#### ## 1. Product

A simple web app that allows students to upload recorded lectures and generate structured notes and a concise summary from the audio.

**\*\*Core flow:\*\***

Upload audio → Process → Generate notes → Edit → Save

---

#### ## 2. Problem

Students may struggle to take complete notes during lectures or may miss classes entirely. When they have a lecture recording, manually listening to it and creating organized notes is time-consuming.

---

#### ## 3. Goal

Help students quickly turn lecture recordings into useful, structured study notes.

---

#### ## 4. Target User

**\*\*Students\*\*** who:

- Record lectures.
- Miss lectures but have recordings.
- Want organized notes from recorded classes.

---

#### ## 5. MVP Scope

##### ### In Scope

- Upload lecture audio.
- Display uploaded file information.
- Process the audio.
- Generate structured lecture notes.
- Generate a lecture summary.

- Edit generated notes.
- Save notes locally.
- View previously saved lectures.
- Delete saved lectures.
- Responsive interface.

### ### Out of Scope

- User accounts
- Cloud storage
- Collaboration
- Social sharing
- Payments
- Flashcards
- Quizzes
- AI chat/tutoring
- Course management
- Analytics

---

## ## 6. User Flow

1. Student opens the app.
2. Student uploads a lecture audio file.
3. App displays the file.
4. Student starts processing.
5. App processes the recording.
6. App generates notes and a summary.
7. Student reviews and edits the content.
8. Student saves the lecture.
9. Student can later reopen or delete it.

---

## ## 7. Functional Requirements

### ### Audio Upload

- User can select a lecture audio file.
- App validates supported audio files.
- App displays the selected file.

### ### Processing

- User can start processing the uploaded audio.
- App shows a processing state.
- App shows an error if processing fails.

### ### Generated Notes

The app should generate:

- Lecture title
- Overview
- Main topics
- Detailed notes
- Key concepts
- Important points
- Examples
- Summary
- Key takeaways

The output should be structured notes, not just a raw transcript.

### ### Editing

Users can edit generated notes before saving.

### ### Persistence

The app uses browser Web Storage to save:

- Lecture information
- Generated notes
- Summary
- User edits
- Creation date

### ### Saved Lectures

Users can:

- View saved lectures.
- Open a saved lecture.
- Edit saved notes.
- Delete a saved lecture.

---

## ## 8. Technical Requirements

- HTML
- CSS
- Vanilla JavaScript
- Web Storage API

- No frameworks
- No backend

AI/transcription functionality should be implemented behind a replaceable processing layer so that mock processing can be used during development and a real AI/transcription service can be integrated later.

---

## ## 9. UI Requirements

The app should have three primary areas:

### ### Dashboard

- Upload lecture action
- List of saved lectures

### ### Processing

- Uploaded file information
- Processing status

### ### Notes

- Generated summary
- Structured notes
- Edit functionality
- Save functionality

The interface should be simple, clean, responsive, and focused on the core workflow.

---

## ## 10. MVP Completion Criteria

The MVP is complete when a student can:

1. Upload a lecture audio file.
2. Process the file.
3. Generate structured notes and a summary.
4. Edit the generated content.
5. Save it to browser storage.
6. Refresh the app and access the saved lecture.
7. Delete the saved lecture.